

Comandi utili in Racket

INDICE

- LISTE: pag 2

- creazione di liste: pag 2
- controllo liste: pag 2
- manipolazione liste: pag 2
- conversione: pag 3
- note sulle liste: pag 4

-STRINGHE: pag 6

- creazione di stringhe: pag 6
- accesso e modifica: pag 6
- confronto e ricerca: pag 6
- conversione: pag 7
- note sulle stringhe: pag 7

-ESPRESSIONI MATEMATICHE: pag 8

-ESEMPI DI CODICI: pag 9

- JAVA: pag 21

convenzioni:

x y z = una serie di caratteri ed elementi qualsiasi (x se va inserito un solo elemento)

xxx = una funzione qualsiasi

n = un numero qualsiasi

#\a #\b #\c = una serie di char qualsiasi

#\a = un char qualsiasi



-liste

creazione di liste

(list x y z) Crea una lista dei suoi argomenti.	(list 1 2 3) ; => '(1 2 3)
(list* x y '(z z1)) Crea una lista concatenando un certo numero di argomenti, l'ultimo dei quali deve essere una lista.	(list* 1 2 '(3 4)) ; => '(1 2 3 4)
(make-list n x) Crea una lista di lunghezza specificata, riempita con un valore.	(make-list 3 0) ; => '(0 0 0)
(build-list n xxx) Crea una lista usando una funzione per generare ciascun elemento.	(build-list 3 add1) ; => '(1 2 3)

Controllo liste

(length '(x y z)) Restituisce la lunghezza di una lista.	(length '(1 2 3)) ; => 3
(null? '(x y z)) Controlla se una lista è vuota.	(null? '()) ; => #f
(list? '(x y z)) Controlla se un valore è una lista.	(list? '(1 2 3)) ; => #t
(andmap xxx '(x y z)) Restituisce vero se il predicato è vero per tutti gli elementi.	(andmap even? '(2 4 6)) ; => #t
(ormap xxx '(x y z)) Restituisce vero se il predicato è vero per almeno un elemento.	(ormap even? '(1 3 4)) ; => #t

Manipolazione di liste

(car '(x y z)) Restituisce il primo elemento di una lista	(car '(1 2 3)) ; => 1
(cdr '(x y z)) Restituisce la lista senza il primo elemento.	(cdr '(1 2 3)) ; => '(2 3)
(cons x '(y z)) aggiunge un elemento all'inizio di una lista	(cons 1 '(2 3)) ; => '(1 2 3)
(append '(x y) '(z))	(append '(1 2) '(3 4)) ; => '(1 2 3 4)

Concatena due o più liste ¹ .	
(reverse ' (x y z)) Inverte una lista.	(reverse '(1 2 3)) ; => '(3 2 1)
(list-ref ' (x y z) n) Restituisce l'elemento in una posizione specifica (0-indexed) ² .	(list-ref '(1 2 3) 1) ; => 2
(map xxx ' (x y z)) Applica una funzione a ciascun elemento di una lista e restituisce una nuova lista.	(map add1 '(1 2 3)) ; => '(2 3 4)
(filter xxx ' (x y z)) Filtra gli elementi di una lista in base a un predicato	(filter even? '(1 2 3 4)) ; => '(2 4)
(foldl xxx ' (x y z))⁴ Riduce una lista da sinistra usando una funzione e un valore iniziale.	(foldl + 0 '(1 2 3)) ; => 6
(foldr xxx ' (x y z)) Riduce una lista da destra usando una funzione e un valore iniziale.	(foldl + 0 '(1 2 3)) ; => 6

conversione³

(string->list "xyz") Converte una stringa in una lista di caratteri.	(string->list "hello") ; => '(#\h #\e #\l #\l #\o)
---	--

note sulle liste:

- 1 non funziona con elementi che non sono strettamente liste (ad esempio numeri) generando il seguente errore, può anche accettare più di due liste alla volta.

append: last argument must be a list, but received x

append: expects a list, given x

- 2 per questa funzione racket incomincia a contare da 0: la prima posizione è 0, la seconda 1 e così via.
- 3 il processo funziona in maniera anche contraria (basta invertire l'ordine prima e dopo la freccia), basta che vengano rispettate le condizioni di input (che si possono vedere osservando l'output precedente)
- 4 produce un output processando tutta la lista con la funzione data, partendo da sinistra (foldr per partire da destra), estremamente utile in funzioni che richiedono calcolo combinatorio, come per simulare il fattoriale (prestando attenzione ad includere 0 come caso particolare):

*(foldl * 1 '(1 2 3 4 5))* da come output 120

```

;1)
(define belong? ;verifica se x è un elemento di S
  (lambda (x S) ;intero x e una lista ordinata di interi S,
    (if (null? S)
      #false
      (if (= x (car S))
        #true
        (belong? x (cdr S)))
      )))
;(belong? 18 '(5 7 10 18 23)) true
;(belong? 18 '(5 7 10 12 23)) false
;(belong? 18 '()) false

;2)
(define position ;restituisce la posizione (indice) di x in S
  (lambda (x S) ;lista senza ripetizioni S
    ;(if (null? S) non serve controllo dato che si assume che x sia un elemento di S.
    (+ (if (= x (car S))
      0
      (+ 1 (position x (cdr S)))))
    ))

;(position 7 '(7 8 24 35 41)) -> 0
;(position 35 '(7 8 24 35 41)) -> 3
;(position 41 '(7 8 24 35 41)) -> 4

```

```

(define sorted-ins ;restituisce la lista ordinata/no ripetizioni che contiene x e tutti gli elem di S
  (lambda (x S) ;lista senza ripetizioni S
    (if (null? S)
      (cons x S)
      (if (belong? x S)
        S ;x c'è già
        (if (< x (car S))
          (cons x S)
          (cons (car S) (sorted-ins x (cdr S))))
        )))
;(sorted-ins 24 '()); -> '(24)
;(sorted-ins 5 '(7 8 24 35 41)); -> '(5 7 8 24 35 41)
;(sorted-ins 24 '(7 8 24 35 41)); -> '(7 8 24 35 41)
;(sorted-ins 27 '(7 8 24 35 41)); -> '(7 8 24 27 35 41)

```

```

(define delega
  (lambda (l1 l2) ;l2 '(), l1 data da input
    (if (null? (cdr l1))
      (sorted-ins (car l1) l2)
      (delega (cdr l1) (sorted-ins (car l1) l2)))))
(define sorted-list ;restituisce la lista ordinata e senza ripetizioni S' che contiene tutti e soli gli interi di S.
  (lambda (S) ;lista di interi non necess ordinata e con possibili doppiioni
    (if (null? S)
      "non ho ricevuto una lista da ordinare"
      (delega S '())))
)
;(sorted-list '())
;(sorted-list '(35 8 41 24 7)) ;-> '(7 8 24 35 41)
;(sorted-list '(35 24 8 41 24 7 44)) ;-> '(7 8 24 35 41 44)
;(sorted-list '(-3 2 5 87))

```

Funzionamento doppio-lambda / map (differenza tra inline e con funzione)

(sono 3 esempi separati)

```
1 (define make-adder
2   (lambda (x)                ; x è il parametro per il lambda esterno
3     (lambda (y)              ; y è il parametro per il lambda interno
4       (+ x y))))             ; restituisce la somma di x e y
5 (define adder (make-adder 5)) ; adder è una funzione che aggiunge 5
6
7 ;(adder 10) ; Risultato: 15
8 ;(adder 20) ; Risultato: 25
9 ;il 5 è la x, mentre il 10 e 20 sono la y
10
11 (define tas
12   (lambda (n)
13     (cond ((= n 0)
14            '(()))
15           ((= n 1)
16            '((#\R) (#\B)))
17           (else
18            (append
19              (map (lambda (x) (append '(\R \B) x)) (tas (- n 2)))
20              ;identico a (map (add-tiles '(\R \B)) (tas (- n 2)))
21              (map (lambda (y) (append '(\B) y)) (tas (- n 1)))
22              ;identico a (map (add-tiles '(\B)) (tas (- n 1)))
23              )))))
24 (define add-tiles
25   (lambda (t)
26     (lambda (l)
27       (append t l)
28       )))
29
30
31 (define app
32   (lambda (x)
33     (lambda (y)
34       (string-append x y)
35       )))
36 (define prova (app "9"))
37
38 (prova "u") ;risultato: "9u"
39
```

-Stringhe

creazione di stringhe

(string #\a #\b #\c) Crea una stringa dai caratteri forniti.	<code>(string #\h #\e #\l #\l #\o) ; => "hello"</code>
(make-string n #\a) Crea una stringa di lunghezza n, riempita con un carattere.	<code>(make-string 5 #\x) ; => "xxxxx"</code>

accesso e modifica

(string-ref "abc" x) Restituisce il carattere in una posizione specifica.	<code>(string-ref "hello" 1) ; => #\e</code>
(substring "abc" x y) Estrae una sottostringa data una posizione iniziale e finale.	<code>(substring "hello" 1 4) ; => "ell"</code> <code>(substring "hello" 2) ; => "llo"</code>
(string-append "abc" " " "a1b1c1") Concatenazione di stringhe.	<code>(string-append "hello" " " "world") ; => "hello world"</code>
(string-upcase "abc") Cambiano il case della stringa.	<code>(string-upcase "hello") ; => "HELLO"</code>
(string-downcase "abc") Cambiano il case della stringa.	<code>(string-downcase "HELLO") ; => "hello"</code>
(string-length "abc") Restituisce la lunghezza di una stringa.	<code>(string-length "hello") ; => 5</code>

confronto e ricerca

(string? "abc") Verifica se un valore è una stringa.	<code>(string? "hello") ; => #t</code>
(string-ci=? "abc" "ABC")² Confrontano le stringhe ignorando le differenze tra maiuscole e minuscole.	<code>(string-ci=? "hello" "HELLO") ; => #t</code>
(string-ci<? "ab" "ABC") Confrontano le stringhe ignorando le differenze tra maiuscole e minuscole.	<code>(string-ci<? "hell" "HELLO") ; => #t</code>
(string-ci>? "abc" "AB") Confrontano le stringhe ignorando le differenze tra maiuscole e minuscole.	<code>(string-ci>? "hello" "HELL") ; => #t</code>
(string<? "abc" "a1b1c1")³	<code>(string<? "apple" "banana") ; => #t</code>

Confrontano le stringhe.	
--------------------------	--

conversione

(string->list "abc") Converte una stringa in una lista di caratteri	(string->list "hello") ; => '(\h \e \l \ #\o)
(list->string '(\a \b \c)) Converte una lista di caratteri in una stringa	(list->string '(\h \e \l \ #\o)) ; => "hello"

note sulle stringhe:

- 1 accetta solo ed esclusivamente caratteri forniti con il formato #\ altrimenti genera l'errore:

string: expects a char
- 2 confronta le due stringhe interrompendosi al primo carattere diverso, che utilizza per determinare se sia #f i #t: il carattere selezionato viene controllato nell'ordine ascii (ad esempio #\a ha come valore 97) e quindi utilizzato come mezzo di confronto in base a se si usava =, < o > (e simili). lo stesso principio si applica a *string=?* e simili, in quel caso tenendo conto se il char sia maiuscolo o minuscolo.
- 3 Per praticità racchiudiamo in questa singola funzione anche *string=?*, *string<?*, *string>?*, *string<=?*, *string>=?* che svolgono la medesima funzione, solo usando un modo di comparazione diverso.

-ESPRESSIONI MATEMATICHE

FUNZIONE	DESCRIZIONE	ESEMPIO	RISULTATO
+	Somma	(+ 3 5)	8
-	Sottrazione	(- 10 4)	6
*	Moltiplicazione	(* 6 7)	42
/	Divisione	(/ 20 4)	5
modulo	Resto della divisione intera	(modulo 10 3)	1
remainder	Resto con segno del divisore	(remainder 10 3)	1
quotient	Quoziente della divisione intera	(quotient 20 3)	6
abs	Valore assoluto	(abs -7)	7
log	Logaritmo naturale	(log 2.71828)	1
log10	Logaritmo in base 10	(log10 1000)	3
exp	Esponenziale (e^x)	(exp 1)	2.71828
expt	Esponenziale	(expt 2 3)	8
sqrt	Radice quadrata	(sqrt 9)	3
floor	Arrotonda per difetto	(floor 4.7)	4
ceiling	Arrotonda per eccesso	(ceiling 4.2)	5
round	Arrotonda al più vicino intero	(round 4.5)	5
truncate	Tronca il valore decimale	(truncate 4.7)	4
=, equal?	Controllo di uguaglianza	(= 5 5)	#t
<, >, <=, >=	Confronti	(< 3 7)	#t
exact->inexact	Converte in numero approssimato	(exact->inexact 3)	3.0

convertire in base 10 un numero in base n (8 nell'esempio)

77345_8 —————

5×8^0	=	5_{10}
4×8^1	=	32_{10}
3×8^2	=	192_{10}
7×8^3	=	3584_{10}
7×8^4	=	28672_{10}

—————> totale 32485_{10}

conversione in base n (nell'esempio base 8) di un numero in base 10

32485_{10} —————

$32485 \div 8$	=	4060 con resto di 5
$4060 \div 8$	=	507 con resto di 4
$507 \div 8$	=	63 con resto di 3
$63 \div 8$	=	7 con resto di 7
$7 \div 8$	=	0 con resto di 7

—————> 77345_8

ESEMPI DI CODICE

```

1  ;; LCS con integrate le posizioni dei simboli posti in corrispondenza
2  ;; Struttura del risultato: lista di terne (simbolo + coppia di indici)
3  ;; Avvio: inizializzazione dei parametri aggiuntivi
4  (define xlcs          ; val: lista di terne
5    (lambda (u v)      ; u, v: stringhe
6      (xlcs-rec u v 0 0)))
7  ;; Riorganizzazione: lcs --> xlcs-rec con parametri aggiuntivi
8  (define xlcs-rec      ; val: lista di terne
9    (lambda (u v i j)  ; u, v: stringhe, i, j: indici
10      (cond
11        ((or (string=? u "") (string=? v ""))
12         null)
13        ((char=? (string-ref u 0) (string-ref v 0))
14         (cons
15          (list (substring u 0 1) i j)
16          (xlcs-rec (substring u 1) (substring v 1) (+ i 1) (+ j 1))))
17        (else
18         (xlonger
19          (xlcs-rec (substring u 1) v (+ i 1) j)
20          (xlcs-rec u (substring v 1) i (+ j 1))
21          )))
22      )))
23
24
25  ;; Confronto di potenziali soluzioni: longer --> xlonger
26
27  (define xlonger        ; val: lista di terne
28    (lambda (s t)        ; s, t: liste di terne
29      (if (< (length s) (length t))
30          t
31          s)))

```

```

;; Lunghezza della sottosequenza comune piu' lunga
;; Length of the Longest Common Subsequence (LLCS)
(define llcs          ; valore: intero
  (lambda (u v) ; u, v: stringhe
    (cond ((or (string=? u "") (string=? v ""))
            0)
          ((char=? (string-ref u 0) (string-ref v 0))
            (+ 1 (llcs (substring u 1) (substring v 1))))
          (else
            (max (llcs (substring u 1) v) (llcs u (substring v 1))))))
;; Sottosequenza comune piu' lunga -- campione
;; Longest Common Subsequence (LCS)
(define lcs          ; valore: stringa
  (lambda (u v) ; u, v: stringhe
    (cond ((or (string=? u "") (string=? v ""))
            "")
          ((char=? (string-ref u 0) (string-ref v 0))
            (string-append (substring u 0 1)
                           (lcs (substring u 1) (substring v 1))))
          (else
            (longer (lcs (substring u 1) v) (lcs u (substring v 1))))))

```

```

(define longer ; valore: stringa
  (lambda (u v) ; u, v: stringhe
    (if (< (string-length u) (string-length v))
        v
        u
    )))
;; Elenco delle sottosequenze comuni piu' lunghe -- tutte
;; Longest Common Subsequences (LCSs)

(define lcss ; valore: stringa
  (lambda (u v) ; u, v: stringhe
    (cond ((or (string=? u "") (string=? v ""))
      (list ""))
      ((char=? (string-ref u 0) (string-ref v 0))
      (prefix-all (substring u 0 1)
        (lcss (substring u 1) (substring v 1))))
      (else
      (let ((m (llcs (substring u 1) v))
            (n (llcs u (substring v 1))))
        )
      (cond ((< m n)
        (lcss u (substring v 1)))
        ((> m n)
        (lcss (substring u 1) v))
        (else
        (merge (lcss (substring u 1) v) (lcss u (substring v 1))))
        )))
    )))

;; Procedure di supporto

(define prefix-all ; val: lista di stringhe
  (lambda (pre words) ; pre: stringa, words: lista di stringhe
    (if (null? words)
        null
        (cons (string-append pre (car words))
          (prefix-all pre (cdr words))))
    )))

(define merge ; val: lista di stringhe
  (lambda (ws1 ws2) ; ws1, ws2: liste di stringhe
    (if (null? ws1)
        ws2
        (merge (cdr ws1)
          (ins (car ws1) ws2)))
    )))

(define ins ; val: lista di stringhe
  (lambda (w words) ; w: stringa, words: lista di stringhe
    (cond ((null? words)
      (list w))
      ((string=? w (car words))
      words)
    )))

```

```

    (else
      (cons (car words) (ins w (cdr words))))
  )))

```

"

"

```

1  (define tassell-1-2
2    (lambda (n)
3      (cond ((= n 1)
4              1)
5            ((= n 2)
6              2)
7            (else
8              (+ (tassell-1-2 (- n 1))
9                (tassell-1-2 (- n 2)))
10             )
11          )))
12
13 (define tassell-b-r
14   (lambda (n)
15     (cond ((= n 1)
16             2)
17           ((= n 2)
18             3)
19           (else
20             (+ (tassell-b-r (- n 1))
21               (tassell-b-r (- n 2)))
22             )
23           )))

```

il seguente codice (tassell-1-2 n) permette di calcolare quante possibili combinazioni di piastrelle di lunghezza 1 unità e 2 unità si possono ottenere dato un pavimento lungo N unità.

il seguente codice (tassell-b-r n) permette di calcolare quante combinazioni possibili di piastrelle (di due colori diversi) di dimensione uguale (una unità) si possono ottenere dato un corridoio lungo n unità

```

1 (define low ;prende come input una lista con all'interno delle stringhe
2 (lambda (l) ;ritorna l'input con il primo carattere sempre minuscolo
3 (if (null? l) '()
4 (cons (string-append (mod (string-ref (car l) 0)) (substring (car l) 1))
5 (low (cdr l))))))
6
7 (define mod
8 (lambda (c)
9 (if (< (char->integer c) 97)
10 (string
11 (integer->char (+ (char->integer c) 32)))
12 (string c)
13 )))
14
15 (low '("Abete" "Betulla" "Faggio" "Quercia" "Tiglio"))
16 ;("abete" "betulla" "faggio" "quercia" "tiglio")
17 (low '("BiancoSpino" "pervinca" "Primula" "RODODENDRO" "Viola"))
18 ;("biancoSpino" "pervinca" "primula" "rododendro" "viola")

```

```

1 ;; Moltiplicazione "del contadino russo":
2 ;; Procedura che applica la ricorsione di coda
3 (define paesant-mul ; val: intero
4 (lambda (m n) ; m, n: interi non negativi
5 (paesant-rec m n 0)
6 ))
7
8 (define paesant-rec ; val: intero
9 (lambda (m n p) ; m, n, p: interi non negativi
10 (cond
11 ((= n 0)
12 p)
13 ((even? n)
14 (paesant-rec (* 2 m) (quotient n 2) p))
15 (else
16 (paesant-rec (* 2 m) (quotient n 2) (+ p m)))
17 )
18 ))

```

questo programma permette di eseguire moltiplicazioni con il metodo del contadino russo

Alcune tracce d'esame

Date due funzioni $f, g : D \rightarrow D$, un elemento $x \in D$ e un intero $k > 0$, la procedura `iter-alt` restituisce una lista di k elementi di D , ciascuno dei quali risulta da successive applicazioni delle funzioni f e g , alternativamente, a partire da x come illustrato dal seguente schema di valutazione — dove nell'ultimo elemento, al termine della sequenza evocata dai puntini, le funzioni vengono applicate complessivamente $k-1$ volte:

$$(\text{iter-alt } f \ g \ x \ k) \rightarrow (x \ f(x) \ g(f(x)) \ f(g(f(x))) \ g(f(g(f(x)))) \ \dots)$$

Detto altrimenti, partendo da x ogni ulteriore elemento si ottiene applicando f oppure g , a turno, al precedente. Esempi:

$$(\text{iter-alt } \text{cdr} \ \text{reverse} \ '(1 \ 2 \ 3 \ 4) \ 5) \rightarrow ((1 \ 2 \ 3 \ 4) \ (2 \ 3 \ 4) \ (4 \ 3 \ 2) \ (3 \ 2) \ (2 \ 3))$$
$$\begin{aligned} (\text{iter-alt} \ (\text{lambda} \ (x) \ (+ \ x \ 1)) \ (\text{lambda} \ (x) \ (/ \ 1 \ x)) \ 1 \ 10) \\ \rightarrow (1 \ 2 \ \frac{1}{2} \ \frac{3}{2} \ \frac{2}{3} \ \frac{5}{3} \ \frac{3}{5} \ \frac{8}{5} \ \frac{5}{8} \ \frac{13}{8}) \end{aligned}$$

Scrivi un programma per realizzare la procedura `iter-alt`.

```
1 (define iter-alt
2   (lambda (f g x k)
3     (if (= k 0) null
4         (cons x (iter-alt g f (f x) (- k 1))))))
```

1. Procedure con argomenti procedurali

Sulla base del programma impostato nel riquadro, la procedura `bin-trees` restituisce la lista di tutti gli alberi binari di struttura diversa con n foglie. Ciascun albero binario è codificato da una stringa in modo analogo a quanto visto in relazione al progetto Huffman: i nodi intermedi sono rappresentati dal simbolo `@` e le foglie dalla lettera `f`, senza distinzioni. Più precisamente, un albero costituito da un solo nodo è rappresentato dalla stringa `"f"`; un albero che si compone di una radice con sottoalbero sinistro codificato dalla stringa `"σ"` e con sottoalbero destro codificato da `"δ"` è rappresentato dalla giustapposizione `"@σδ"`. Esempi:

```
(bin-trees 1) → ("f")                                (bin-trees 2) → ("@ff")
(bin-trees 3) → ("@f@ff" "@@ffff")
(bin-trees 4) → ("@f@f@ff" "@f@@ffff" "@@ff@ff" "@@f@fff" "@@@ffff")
(bin-trees 5) → ("@f@f@f@ff" "@f@f@ffff" "@f@ff@ff" "@f@ff@fff" "@f@fff@fff"
                 "@ff@f@ff" "@ff@ffff" "@ff@ff@ff" "@ff@fff@ff" "@ff@fff@fff"
                 "@fff@ffff" "@fff@ff@ff" "@fff@ff@fff" "@fff@fff@fff")
```

Completa il programma inserendo espressioni appropriate negli spazi indicati.

```
(define bin-trees ; val: lista degli alberi binari di struttura diversa con n foglie
  (lambda (n) ; n>0 intero
    (if (= n 1)
        (list "f")
        (in-lists-merge
         (map (lambda (k) (all-trees-from (bin-trees k) (bin-trees (- n k))))
              (int-range 1 (- n 1))) ; (list 1,2,...,n-1)
         )
        )
    )
  )

(define all-trees-from ; val: lista degli alberi binari con sottoalberi sinistro in lst e destro in rst
  (lambda (lst rst) ; lst, rst: liste di alberi binari
    (in-lists-merge
     (map (lambda (u) (map (lambda (t) (string-append "@" (string-append u t))) rst)) lst) ; in input ho due liste di stringhe
     ; qui come risultato devo avere una lista di liste, ciascun elemento deve essere una lista con una stringa
     ; che ha giustapposto ciascun lst a ciascun rst
     )
    )
  )

(define in-lists-merge ; val: lista risultante dalla fusione (append) delle liste-elementi di v
  (lambda (v) ; v: lista di liste
    (if (null? v)
        null
        (if (null? (cdr v)) ; se ha un solo elemento
            (car v)
            (append ; se ha almeno 2 elementi
                     (append (car v) (car (cdr v))) ; appendo il primo elemento di v con il secondo
                     (in-lists-merge (cdr (cdr v))) ; ricorsione sul resto della lista (dal terzo elemento in poi)
                     )
            )
        )
    )
  )
```

Binary	Oct	Dec	Hex	Glyph
010 0000	040	32	20	
010 0001	041	33	21	!
010 0010	042	34	22	"
010 0011	043	35	23	#
010 0100	044	36	24	\$
010 0101	045	37	25	%
010 0110	046	38	26	&
010 0111	047	39	27	'
010 1000	050	40	28	(
010 1001	051	41	29	)
010 1010	052	42	2A	*
010 1011	053	43	2B	+
010 1100	054	44	2C	,
010 1101	055	45	2D	-
010 1110	056	46	2E	.
010 1111	057	47	2F	/
011 0000	060	48	30	0
011 0001	061	49	31	1
011 0010	062	50	32	2
011 0011	063	51	33	3
011 0100	064	52	34	4
011 0101	065	53	35	5
011 0110	066	54	36	6
011 0111	067	55	37	7
011 1000	070	56	38	8
011 1001	071	57	39	9
011 1010	072	58	3A	:
011 1011	073	59	3B	;
011 1100	074	60	3C	<
011 1101	075	61	3D	=
011 1110	076	62	3E	>
011 1111	077	63	3F	?

Binary	Oct	Dec	Hex	Glyph
100 0000	100	64	40	@
100 0001	101	65	41	A
100 0010	102	66	42	B
100 0011	103	67	43	C
100 0100	104	68	44	D
100 0101	105	69	45	E
100 0110	106	70	46	F
100 0111	107	71	47	G
100 1000	110	72	48	H
100 1001	111	73	49	I
100 1010	112	74	4A	J
100 1011	113	75	4B	K
100 1100	114	76	4C	L
100 1101	115	77	4D	M
100 1110	116	78	4E	N
100 1111	117	79	4F	O
101 0000	120	80	50	P
101 0001	121	81	51	Q
101 0010	122	82	52	R
101 0011	123	83	53	S
101 0100	124	84	54	T
101 0101	125	85	55	U
101 0110	126	86	56	V
101 0111	127	87	57	W
101 1000	130	88	58	X
101 1001	131	89	59	Y
101 1010	132	90	5A	Z
101 1011	133	91	5B	[
101 1100	134	92	5C	\
101 1101	135	93	5D	]
101 1110	136	94	5E	^
101 1111	137	95	5F	_

Binary	Oct	Dec	Hex	Glyph
110 0000	140	96	60	`
110 0001	141	97	61	a
110 0010	142	98	62	b
110 0011	143	99	63	c
110 0100	144	100	64	d
110 0101	145	101	65	e
110 0110	146	102	66	f
110 0111	147	103	67	g
110 1000	150	104	68	h
110 1001	151	105	69	i
110 1010	152	106	6A	j
110 1011	153	107	6B	k
110 1100	154	108	6C	l
110 1101	155	109	6D	m
110 1110	156	110	6E	n
110 1111	157	111	6F	o
111 0000	160	112	70	p
111 0001	161	113	71	q
111 0010	162	114	72	r
111 0011	163	115	73	s
111 0100	164	116	74	t
111 0101	165	117	75	u
111 0110	166	118	76	v
111 0111	167	119	77	w
111 1000	170	120	78	x
111 1001	171	121	79	y
111 1010	172	122	7A	z
111 1011	173	123	7B	{
111 1100	174	124	7C	
111 1101	175	125	7D	}
111 1110	176	126	7E	~

COME RISOLVERE PER INDUZIONE UN PROBLEMA

$\forall x \in S$ P vale per x - individuare il coordinamento su S - individuare la presenza di uno/più elementi più piccoli di S - si deve dimostrare che P vale per tutti gli elementi più piccoli (detti minimali) di S - dimostrare il passo induttivo fissiamo un elemento qualsiasi di S assumiamo che valga l'ipotesi induttiva $\forall x' \in S, x' < x$ P vale su x' verifichiamo che P vale per x	con S si intende un insieme di numeri/una lista/stringhe/coppie... si deve cercare di ordinare l'imput, un esempio banale sarebbe ordinare l'insieme dei numeri reali in ordine crescente essi costituiscono il caso base della dimostrazione, ad esempio per l'insieme $\mathbb{N}$ l'elemento più piccolo è lo 0 L'elemento preso non può essere un elemento preciso e successivamente usato per fare assunzioni date le sue proprietà singole (x ≠ da tutti gli elementi minimali di S - nel caso di $x \in \mathbb{N}, x \neq 0$) ad un certo punto si dovrà usare l'ipotesi induttiva
Correttezza di Unknown vogliamo dimostrare che $\forall n \in \mathbb{N} \text{ (unknown } \underline{n}) \rightarrow \underline{n}^2$ - individuazione caso base $(n = 0)(\text{Unknown } 0) \rightarrow 0^2=0$ $(\text{Unknown } 0) \rightarrow$ $(\text{lambd}a(x)$ $(\text{if } (= x 0) 0$ $(\dots))) \rightarrow$ $(\text{if } (= 0 0) 0$ $(\dots)) \rightarrow 0$	si sottolinea un elemento per indicare che non si tratti di una variabile in scheme il caso base è verificato all'interno del programma

- passo induttivo fisso $n \in \mathbb{N}$, $n \neq 0$ assumo $\forall n' \in \mathbb{N}$, $n' < n$ (unknown $\underline{n}'$) $\rightarrow \underline{n}'^2$ - verificare (unknown $\underline{n}$) $\rightarrow \underline{n}^2$ (unknown n) $\rightarrow$ (if (= n 0) 0 (+)) $\rightarrow$ (+(unknown (- $\underline{n}$ 1)(odd $\underline{n}$)) $\rightarrow$ (+ ($\underline{n}$-1)² (odd $\underline{n}$)) $\rightarrow$ con una veloce risoluzione matematica si trova che il risultato sia = n^2, dimostrando per induzione l'ipotesi	essendo $\underline{n} \neq 0$ per ipotesi si passa al ramo else. la prima espressione valutata è (- n 1) perchè è ancora riducibile (è preferibile usare n + 1) andrebbe dimostrato che $\underline{n}-1 \in \mathbb{N}$, tuttavia in questo caso sarebbe banale in quanto $n \neq 0$ e sottraendo 1 a qualsiasi $n \in \mathbb{N}$ (con $n \neq 0$) si ottiene sempre un $n' \in \mathbb{N}$ con odd si intende 2(n-1)
---	--

```

1 (define manh          ;val: int
2   (lambda (i j)      ;i, j: interi non negativi
3     (if (or (= i 0) (= j 0))
4       1
5       (+ (manh (- i 1) j) (manh i (- j 1)))))
6   )
7 )
8 )
9
10 ; ogni i, j naturali . (manh i j) -> (i+j)! / (i! * j!) (G)
11 ; dimostrazione per induzione su k = i+j
12 ; (0,0) < (0,1), (1,0) < (0,2), (1,1), (2,0) < (0,3), (1,2), (2,1), (3,0) < ...
13 ; (G): ogni k naturale . ogni i, j naturali tali che i+j = k . (manh i j) -> (i+j)! / (i! * j!)
14 ; caso/i base:
15 ; ogni i, j naturali tali che i+j = 0 . (manh i j) -> (i+j)! / (i! * j!)
16 ; (manh 0 0) -> (0+0)! / (0! * 0!) = 1
17 ; ipotesi induttiva: considero un particolare k naturale e assumo
18 ; ogni i, j naturali tali che i+j = k . (manh i j) -> (i+j)! / (i! * j!)
19 ; passo induttivo: con riferimento al k considerato sopra vorrei dimostrare che
20 ; ogni i, j naturali tali che i+j = k+1 . (manh i j) -> (i+j)! / (i! * j!)
21 ; dimostrazione del passo induttivo: ogni i, j naturali tali che i+j = k+1
22 ; (manh i j) -> (if (or (= i 0) (= j 0)) ... ...)
23 ; (a) i = 0, (b) j = 0
24 ; (manh i j) -> (if (or (= i 0) (= j 0)) ... ...) -> 1 = (0+j)! / (0! * j!) = (i+0)! / (i! * 0!)
25 ; (c) i, j > 0
26 ; (manh i j) -> (if (or (= i 0) (= j 0)) ... ...) -> (+ (manh (- i 1) j) (manh i (- j 1)))
27 ; -> (+ (manh i-1 j) (manh i (- j 1))) ; i-1 + j = i+j - 1 = k+1 - 1 = k
28 ; -> (+ ((i-1+j)! / ((i-1)! * j!)) (manh i (- j 1))) ; perciò si applica l'ipotesi induttiva
29 ; -> (+ ((i-1+j)! / ((i-1)! * j!)) (manh i j-1)) ; come sopra
30 ; -> (+ ((i-1+j)! / ((i-1)! * j!)) (i+j-1)! / (i! * (j-1)!))
31 ; -> ((i-1+j)! / ((i-1)! * j!)) + (i+j-1)! / (i! * (j-1)!))

```

```

(define hanoi-picture
  (lambda (n k)
    (h-d n n k '() '() '() (list 1 0) (list 2 0) (list 3 0))))

(define h-d
  (lambda (nFisso n k a b c a1 b1 c1) ; n-numero di dischi, k-iterazione,
    (if (= n 0)
        (compila nFisso a1 b1 c1 a b c)
        (if (< k (expt 2 (- n 1))) ; k < 2^(n-1)
            (h-d nFisso (- n 1) k
                (cons (disk-image n nFisso (car a1) (cadr a1)) a) c b
                (list (car a1) (+ 1 (cadr a1))) c1 b1)
            (h-d nFisso (- n 1) (- k (expt 2 (- n 1)))
                c (cons (disk-image n nFisso (car b1) (cadr b1)) b) a
                c1 (list (car b1) (+ 1 (cadr b1))) a1)
        ))))

```

THEORY

The Elements of Programming

Every powerful language has:

- Primitive expression, which represent the simplest entities the language is concerned with.
- Means of combination, by which compound elements are built from simpler ones.
- Means of abstraction, by which compound elements can be named and manipulated as units.

In programming we deal with two kinds of elements: procedures and data, data is "stuff" that we want to manipulate, and procedures are descriptions of the rules for manipulating the data.

Normal-order evaluation: "fully expand and then reduce method"

Applicative-order evaluation: "evaluate the arguments and then apply"

Procedures, as introduced above, are much like ordinary mathematical functions. They specify a value that is determined by one or more parameters.

The contrast between function and procedure is a reflection of the general distinction between describing properties of things and describing how to do things. In mathematics we are usually concerned with declarative (what is) descriptions, whereas in computer science we are usually concerned with imperative (how to) descriptions.

Iteration can be accomplished using no special construct other than the ordinary ability to call a procedure.

Embedded definitions must come first in a procedure body.

RECURSION AND ITERATION

A recursive process is a chain of deferred operations.

An iterative process is one whose state can be summarized by a fixed number of state variables, together with a fixed rule that describes how the state variables should be updated as the process moves from state to state and an (optional) end test that specifies conditions under which the process should terminate.

In the iterative case, the program variables provide a complete description of the state of the process at any point. If we stopped the computation between steps, all we would need to do to resume the computation is to supply the interpreter with the values of the program variables.

Not so with the recursive process. In this case there is some additional "hidden" information, maintained by the interpreter and not contained in the program variables, which indicates "where the process is" in negotiating the chain of deferred operations.

Scheme will execute an iterative process in constant space, even if the iterative process is described by a recursive procedure. An implementation with this property is called tail-recursive.

JAVA

ESEMPI PRINCIPALI DI CODICE

// File: JavaKeywordsDemo.java

// Definizione del package (organizzazione del codice)

package demo;

// Import di una libreria

import java.util.ArrayList;

// Dichiarazione di classe

public class JavaKeywordsDemo {

 // Variabili di diversi tipi

 int numero = 10;

 double decimale = 3.14;

 boolean attivo = true;

 char lettera = 'A';

 String testo = "Ciao Java!"; // String non è una keyword ma si usa spesso

 // Costante con 'final'

 final int COSTANTE = 100;

```

// Metodo principale

public static void main(String[] args) {

    JavaKeywordsDemo demo = new JavaKeywordsDemo(); // uso di 'new'


    // ----- Condizionali -----

    if (demo.numero > 0) {

        System.out.println("Numero positivo");

    } else {

        System.out.println("Numero non positivo");

    }


    // Switch-case

    int giorno = 3;

    switch (giorno) {

        case 1: System.out.println("Lunedì"); break;

        case 2: System.out.println("Martedì"); break;

        default: System.out.println("Altro giorno");

    }


    // ----- Cicli -----

    for (int i = 0; i < 3; i++) {

        System.out.println("For loop: " + i);

    }


    int x = 0;

    while (x < 3) {

```

```

    System.out.println("While loop: " + x);

    x++;
}

do {

    System.out.println("Do-while loop: " + x);

    x++;
} while (x < 5);


// ----- Eccezioni -----

try {

    int a = 5 / 0; // errore divisione per zero
} catch (ArithmeticException e) {

    System.out.println("Eccezione: " + e.getMessage());
} finally {

    System.out.println("Blocco finally eseguito sempre");
}


// ----- ArrayList (uso con import) -----

ArrayList<String> lista = new ArrayList<>();

lista.add("Uno");

lista.add("Due");

System.out.println("Lista: " + lista);


// ----- Ereditarietà e polimorfismo -----

Animale cane = new Cane();

```

```
        cane.verso(); // uso di override
    }
}
```

// Classe astratta

```
abstract class Animale {
    abstract void verso(); // metodo astratto
}
```

// Classe che estende un'altra

```
class Cane extends Animale {
    @Override
    void verso() {
        System.out.println("Bau!");
    }
}
```

// Interfaccia

```
interface Guidabile {
    void guida();
}
```

// Classe che implementa un'interfaccia

```
class Auto implements Guidabile {
    public void guida() {
        System.out.println("Sto guidando l'auto");
    }
}
```



```
}  
}
```

```
// Enum
```

```
enum Colore {  
    ROSSO, VERDE, BLU  
}
```

MEMOIZATION

// Approccio imperativo: Tecniche di memoization e programmazione dinamica

```
import java.math.*; // BigInteger
```

```
public class Memoization {  
    /* Numeri di Fibonacci (tree recursion)  
    (define fibonacci  
    (lambda (n)      ; n naturale  
    (if (< n 2)  
        1  
        (+ (fibonacci (- n 2)) (fibonacci (- n 1))))  
    )  
    ))  
    */
```

// Traduzione in Java:

```
public static long fibonacci( int n ) { // n >= 0  
    if ( n < 2 )  
        return 1;  
    else  
        return fibonacci( n-2 ) + fibonacci( n-1 );  
}
```

// Nuova versione che applica la tecnica di memoization:

```
public static long fibonacciMem( int n ) { // n >= 0
```

```

    long[] mem = new long[ n+1 ];
    for ( int i=0; i<=n; i=i+1 )
        mem[i] = UNKNOWN;
    return fibonacciRec( n, mem );
}

```

```

private static long fibonacciRec( int n, long[] mem ) {
    if ( mem[n] == UNKNOWN ) {
        if ( n < 2 )
            mem[n] = 1;
        else
            mem[n] = fibonacciRec( n-2, mem ) + fibonacciRec( n-1, mem );
    }
    return mem[n];
}

```

```

private static final long UNKNOWN = -1;

```

// Superamento dei limiti delle rappresentazioni numeriche a parola fissa:

// int: risultato rappresentabile per n <= 45

// long: risultato rappresentabile per n <= 91

// BigInteger: rappresentazione intera equiparabile a quella di Scheme

```

public static BigInteger fibonacciBig( int n ) { // n >= 0

```

```

    BigInteger[] mem = new BigInteger[ n+1 ];

```

```

    for ( int i=0; i<=n; i=i+1 )

```

```

        mem[i] = BIG_UNKNOWN;

```

```

    return fibonacciBigRec( n, mem );
}

```

```
}
```

```
private static BigInteger fibonacciBigRec( int n, BigInteger[] mem ) {  
    if ( mem[n] == BIG_UNKNOWN ) {  
        if ( n < 2 )  
            mem[n] = new BigInteger( "1" );  
        else  
            mem[n] = fibonacciBigRec( n-2, mem ).add( fibonacciBigRec(n-1,mem) );  
    }  
    return mem[n];  
}
```

```
private static final BigInteger BIG_UNKNOWN = null;
```

// Diversa versione che applica una tecnica di programmazione dinamica:

```
public static long fibonacciDp( int n ) { // n >= 0  
    long[] mem = new long[ n+1 ];  
    mem[0] = 1;  
    if ( n > 0 )  
        mem[1] = 1;  
    for ( int k=2; k<=n; k=k+1 )  
        mem[k] = mem[k-2] + mem[k-1];  
    return mem[n];  
}
```

/* Problema dei "Percorsi di Manhattan"

(define manhattan

(lambda (i j) ; i, j naturali

```

(if (or (= i 0) (= j 0))
  1
  (+ (manhattan (- i 1) j) (manhattan i (- j 1))))
)
))
*/

```

// Traduzione in Java:

```

public static long manhattan( int i, int j ) { // i, j >= 0
    if ( (i == 0) || (j == 0) )
        return 1;
    else
        return manhattan( i-1, j ) + manhattan( i, j-1 );
}

```

// Nuova versione che applica la tecnica di memoization:

```

public static long manhattanMem( int i, int j ) { // i, j >= 0
    long[][] mem = new long[ i+1 ][ j+1 ];
    for ( int u=0; u<=i; u=u+1 ) {
        for ( int v=0; v<=j; v=v+1 )
            mem[u][v] = UNKNOWN;
    }
    return manhattanRec( i,j, mem );
}

```

```

private static long manhattanRec( int i, int j, long[][] mem ) {
    if ( mem[i][j] == UNKNOWN ) {

```

```

    if ( (i == 0) || (j == 0) )
        mem[i][j] = 1;
    else
        mem[i][j] = manhattanRec( i-1, j, mem ) + manhattanRec( i, j-1, mem );
    }
    return mem[i][j];
}

```

// Altra versione che applica una tecnica di programmazione dinamica:

```

public static long manhattanDp( int i, int j ) { // i, j >= 0

    long[][] mem = new long[i+1][j+1];    // Economia di spazio di memoria

    for ( int y=0; y<=j; y=y+1 )    // manhattan(0,v), v in [0,j]
        mem[0][y] = 1;

    for ( int x=0; x<=i; x=x+1 )    // manhattan(0,v), v in [0,j]
        mem[x][0] = 1;

    for ( int x=1; x<=i; x=x+1 ) {
        for ( int y=1; y<=j; y=y+1 ) // manhattan(u,v), v in [0,j]
            mem[x][y] = mem[x-1][y] + mem[x][y-1];
    }

    return mem[i][j];                // manhattan(i,j)
}

```

//lscs

/**

* (define llcs ;val: int

* (lambda (u,v) ;u,v : stringhe

* (cond ((or(string=? u "")(string=? v ""))

* 0

```

*      )
*      ((char=? (string-ref u 0)(string-ref v 0))
*      (+ 1 (llcs (substring u 1)(substring v 1))))
*      )
*      (else
*      (max (llcs (substring u 1) v)(llcs u (substring v 1))))
*      )
*      )
*  ))
*/

```

```

private static final int UNKNOWN_INT = -1;

```

```

public static int llcs( String u, String v) {

```

```

    int m=u.length();

```

```

    int n=v.length();

```

```

    int[][] mem = new int[m+1][n+1];

```

```

    for ( int i=0; i<=m; i=i+1 ) {

```

```

        for ( int j=0; j<=n; j=j+1 )

```

```

            mem[i][j] = UNKNOWN_INT;

```

```

        }

```

```

        return llcsRec( u,v, mem );

```

```

    }

```

```

private static int llcsRec( String u, String v, int[][] mem ) {

```

```

    int i=u.length();

```

```

int j=v.length();
if ( mem[i][j] == UNKNOWN ) {
    if ( (i == 0) || (j == 0) )
        mem[i][j] = 0;
    else if(u.charAt(0)==v.charAt(0))
        mem[i][j]=1+llcsRec(u.substring(1),v.substring(1),mem);
    else{
        mem[i][j] = Math.max(llcsRec(u.substring(1),v,mem),
            llcsRec(u,v.substring(1),mem));
    }
}
return mem[i][j];
}

```

//lcs con le stringhe

```

public static String lcs( String u, String v) {

    int m=u.length();
    int n=v.length();
    String[][] mem = new String[m+1][n+1];

    for ( int i=0; i<=m; i=i+1 ) {
        for ( int j=0; j<=n; j=j+1 )
            mem[i][j] = null;
    }
    return lcsRec( u,v, mem );
}

```



```

private static String lcsRec( String u, String v, String[][] mem ) {
    int i=u.length();
    int j=v.length();
    if ( mem[i][j] == null ) {
        if ( (i == 0) || (j == 0) )
            mem[i][j] = "";
        else if(u.charAt(0)==v.charAt(0))
            mem[i][j]=u.substring(0,1)+lcsRec(u.substring(1),v.substring(1),mem);
        else{
            mem[i][j] = longer(lcsRec(u.substring(1),v,mem),
                               lcsRec(u,v.substring(1),mem));
        }
    }
    return mem[i][j];
}

```

```

private static String longer(String u, String v){
    int m=u.length();
    int n=v.length();
    if(m<n)
        return v;
    else if (m>n)
        return u;
    else if(Math.random()<0.5)
        return v;
    else
        return u;
}

```

```

//lcsDP versione migliore

public static String lcsDP( String u, String v) {

    int m=u.length();
    int n=v.length();
    int[][] mem = new int[m+1][n+1];

    for ( int j=0; j<=n; j++ )
        mem[0][j] =0;
    for ( int i=1; i<=m; i++ )
        mem[i][0] =0;
    for ( int i=1; i<=m; i++) {
        for ( int j=1; j<=n; j++ ) {
            if(u.charAt(m-i)==v.charAt(n-j))
                mem[i][j] =1+mem[i-1][j-1];
            else
                mem[i][j] =Math.max(mem[i-1][j],mem[i][j-1]);
        }
    }

    String s = ""; //uguale a String s= new String("")
    int i=m;
    int j=n;

    while(mem[i][j]>0){
        if(u.charAt(m-i)==v.charAt(n-j)){
            s=s+u.charAt(m-i); //è uguale all'append

```

```

        i--;
        j--;
    }else if(mem[i-1][j]<mem[i][j-1])
        j--;
    else if(mem[i-1][j]<mem[i][j-1])
        i--;
    else if(Math.random()<0.5)
        j--;
    else
        i--;
    }
    return s;
}
} // class Memoization

```

```

import java.util.*;

/**
 * 19-06-24.pdf
 *
 * balancedSum(new int[] {2,2,5,2,4,1,2,4,2,1,2,3})-->15
 */

public class soluzioni
{
    public static int balancedSum(int[] s){
        int n = s.length;

        for(int i=0; i<n; i = i+1){
            int sin = 0;
            int des = 0;
            for(int j=0; j<n; j=j+1){
                if(j<i)
                    sin=sin+s[j];
                else
                    des=des+s[j];
            }
            if(sin == des)
                return sin;
        }
        return 0;
    }

    /**
     * 2. Programmazione dinamica bottom-up

```

Il seguente programma ricorsivo determina il numero di Stirling di II specie di indici interi n e k :

```
sti(n,k) = {  
    n  
    k } per  $1 \leq k \leq n$  ("problema dei pasticcini")  
public static long sti( int n, int k ) { //  $1 \leq k \leq n$   
  
    if ( (k == 1) || (k == n) ) {  
        return 1;  
    } else {  
        return sti(n-1,k-1) + k*sti(n-1,k);  
    }  
}
```

Il programma impostato nel riquadro applica una tecnica bottom-up di programmazione dinamica per rendere più

efficiente la computazione ricorsiva di sti. Ispirandosi ad esempi analoghi discussi a lezione, si potrebbe utilizzare

una matrice bidimensionale per registrare i valori delle invocazioni ricorsive di sti(n,k) in corrispondenza agli indici

$\langle n, k \rangle$. Tuttavia, si può anche osservare che i valori di una riga di indice $n > 1$ dipendono esclusivamente dai valori

Ciò suggerisce di fare economia di memoria, conservando solo i valori di una riga

in un array unidimensionale e sostituendo di volta in volta i valori di una riga "nuova" a quelli della riga precedente

(avendo cura di prestare attenzione al fatto che la maggior parte dei valori registrati vengono utilizzati due volte per

passare alla riga successiva). Il metodo statico nextRow svolge precisamente questa funzione: l'esecuzione di

nextRow(i,mem) ha l'effetto di sostituire nell'array mem i valori della riga di indice i a quelli della riga di indice $i-1$.

Completa la definizione del metodo statico nextRow.

*/

```

public static long sti( int n, int k ) { //  $1 \leq k \leq n$ 

    if ( (k == 1) || (k == n) ) {

        return 1;

    } else {

        return sti(n-1,k-1) + k*sti(n-1,k);

    }

}

```

```

public static long stiDp( int n, int k ) {

    int[] mem = new int[ n+1 ];

    mem[1] = 1;           // sti(1,1) = 1 è l'unico elemento utile della riga di indice 1

    for ( int i=2; i<=n; i=i+1 ) { // prima dell'iterazione per i: mem[j] = sti(i-1,j) per j=1, 2, ..., i-1

        nextRow( i, mem );      // sti(i-1,...) → sti(i,...)

    }                          // dopo l'iterazione relativa a i: mem[j] = sti(i,j) per j=1, 2, ..., i

    return mem[k];            // sti(n,k)

}

```

```

private static void nextRow(int i, int[]mem ) {

    mem[i]=1;

    for(int j=i-1; j>1; j=j-1){

        mem[j] = mem[j-1]+ j*mem[j];

    }

}

```

/**

* Il seguente programma simula il processo risolutivo del rompicapo della Torre di Hanoi utilizzando un oggetto

towers di tipo HanoiTowers che modella le varie configurazioni attraversate nel corso del gioco. In particolare,

l'invocazione del metodo `towers.move(s,d)` consente di spostare un disco dalla cima della torre in corrispondenza

all'asticella `s` alla cima della torre dell'asticella `d`, dove $s, d \in \{0, 1, 2\}$. L'oggetto `towers` registra anche quante volte

un disco viene spostato in corrispondenza a un'asticella "vuota", in cui cioè non sono collocati altri dischi, e tale

informazione può essere acquisita invocando il metodo `towers.count()`. Quindi, il metodo statico `hanoiCount`

restituisce proprio questa informazione al termine del gioco.

codice copiato sotto

Completa la definizione del metodo `hanoiIter` impostato nel riquadro per trasformare il programma ricorsivo in un

corrispondente programma iterativo che applica uno stack.

*/

```
public static int hanoiCount( int n ) {
```

```
    HanoiTowers towers = new HanoiTowers( n );
```

```
    hanoiRec( n, 0, 1, 2, towers );
```

```
    return towers.count();
```

```
}
```

```
private static void hanoiRec( int n, int s, int d, int t, HanoiTowers towers ) {
```

```
    if ( n == 1 ) {
```

```
        towers.move( s, d );
```

```
    } else if ( n > 1 ) {
```

```
        hanoiRec( n-1, s, t, d, towers );
```

```
        hanoiRec( 1, s, d, 0, towers );
```

```
        hanoiRec( n-1, t, d, s, towers );
```

```
    }
```

```
}
```

```

public static int hanoiIter( int n ) {

    HanoiTowers towers = new HanoiTowers( n );

    Stack<int[]> stack = new Stack<int[]>();
    stack.push( new int[] {n,0,1,2} );

    do {
        int[] h = stack.pop();

        if ( h[0]== 1 ) {
            towers.move(h[1],h[2]);
        } else if (h[0]>1) {
            stack.push( new int[] {h[0]-1, h[3], h[2], h[1]} );
            stack.push( new int[] {    1, h[1], h[2],  0 } );
            stack.push( new int[] {h[0]-1, h[1], h[3], h[2]} );
        }
    } while (!stack.empty() );

    return towers.count();
}

```

//es 4

```

public static class HanoiTowers {

    private IntSList[] twr;           // torri rappresentate da liste con il disco in cima
    all'inizio

    private int count;

    public HanoiTowers( int n ) {

```



```

    twr = new IntSList[3];

    // dischi collocati in ciascuna delle tre torri

    for ( int i=0; i<3; i=i+1 ) {      // creazione delle tre torri inizialmente vuote

        twr[i] = IntSList.NULL_INTLIST;

    }

    for ( int j=0; j<n; j=j+1 ) {      // colloca n dischi di diametro n, n-1, ..., 2, 1

        twr[0] = twr[0].cons(n-j);    // in corrispondenza all'asticella 0

    }

    count = 0;                        // spostamenti di dischi verso una asticella "vuota"

}

public boolean empty( int i ) {      // verifica se l'asticella i è "vuota" (senza dischi)

    return twr[i].isNull();

}

public int topDisk( int i ) {        // diametro del disco in cima all'asticella i

    return twr[i].car();

}

public int count() {

    return count;

}

public void move( int i, int j ) {   // spostamento di un disco dall'asticella i a j

    if (empty(j)) {

        count = count + 1;

    }

}

```

```
        twr[j]=twr[j].cons(twr[i].car());  
        twr[i]= twr[i].cdr();  
    }  
} // class HanoiTowers  
}
```

```
/**
```

```
 * esercitazione d'esame
```

```
 * questo programma lo prendi e lo applichi a qualsiasi cosa ti si presenti davanti,  
 estremamente efficiente e pratico
```

```
 */
```

```
public class rec
```

```
{
```

```
    public static long rec(int x, int y, int z){
```

```
        if((x == 1 ) || (y == z)){
```

```
            return 1;
```

```
        }else {return rec ( x-1, y , z) + x * rec(x, y+1, z);
```

```
        }
```

```
    }
```

```
/**
```

```
 * con valori rec(8,5,12) ci si chiede:
```

```
 * valore più piccolo di x = 1
```

```
 * valore più piccolo di y = 5
```

```
 * valore più piccolo di z = 12
```

```
 * valore più grande di x = 8
```

```
 * valore più grande di y = 12
```

```
 * valore più grande di z = 12x
```

```
 *
```

```
 * converti il codice per renderlo più efficiente
```

```
 */
```

```
public static long recPD(int x, int y, int z){
```

```
    long[][] mem = new long[x+1][z+2]; //chiedere se sarebbe più efficiente scrivere [z-y+1]  
    per evitare tutti i valori minori di y non utilizz
```

```

for (int col= y; col<=z-y+1; col=col+1){
    mem[1][col]=1;
}
for(int row=2;row<=y;row=row+1){
    mem[row][x]=1;
}

for(int row=2;row<=x;row=row+1){
    for(int col=z-1; col>=y;col=col-1){
        mem[row][col]=mem[row-1][col]+row*mem[row][col+1];
    }
}
return mem[x][y];
}
}

```

HUFFMAN

```
import huffman_toolkit.*;
```

```
import java.util.*;
```

```
public class Huffman
```

```
{
```

```
    public static int[] charFreq(String src){
```

```
        int[] freq = new int[InputTextFile.CHARS]; //potresti anche mettere 128 come numero  
        fissato
```

```
        for(int c=0; c<freq.length; c++)
```

```
            freq[c]=0;
```

```
        InputTextFile in = new InputTextFile(src); //per provare a capire cosa fa guarda  
        EsempioIO
```

```
        while(in.textAvailable()){
```

```
            char c= in.readChar();
```

```
            freq[c]=freq[c]+1;
```

```
        }
```

```
        in.close();
```

```
        return freq;
```

```
    }
```

```
    public static Node huffmanTree(int[] freq){
```

```

PriorityQueue<Node> queue= new PriorityQueue<Node>();
for(int c=0; c<freq.length; c=c+1){
    if(freq[c]>0){
        Node n = new Node((char) c, freq[c]);
        queue.add(n);
    }
    while(queue.size(>1){
        Node ln = queue.poll();
        Node rn = queue.poll();

        queue.add(new Node(ln,rn));
    }
}
if (queue.isEmpty()) return null;
return queue.poll();
}

```

```

public static String[] huffmanTable(Node n){
    String[] tab = new String[InputTextFile.CHARS];
    Stack<Frame> stack= new Stack<Frame>();
    stack.push(new Frame("",n));

    do{
        Frame f = stack.pop();
        String p = f.path;
        n= f.node;

        if(n.isLeaf()){

```

```

        tab[n.symbol()]=p;
    }else{
        stack.push(new Frame(p + "1", n.right()));
        stack.push(new Frame(p + "0", n.left()));
    }
}while(!stack.empty());

return tab;
}

/* metodi ricorsivi
public static String[] huffmanTable(Node root){
    String[] tab = new String[InputTextFile.CHARS];
    fillTable("", root, tab );
    return tab;
}

public static void fillTable(String p, Node n, String [] tab){

    if(n.isLeaf())
        tab[n.symbol()] = p;
    else{
        fillTable(p+"0", n.left(), tab);
        fillTable(p+"1", n.right(), tab);
    }
}

/* motedodo ricorsivo

```

```

public static String codeTree(Node n){

    if(n.isLeaf()){
        char c = n.symbol();
        if((c=='\\')||(c=='@')){
            return "\\" + c;
        }else{
            return "" + c;
        }
    }else{
        return "@" + codeTree(n.left())+codeTree(n.right());
    }
}

```

```

private static Node retrieveTree( InputTextFile in){
    char c= in.readChar();
    if(c=='@'){
        Node ln= retrieveTree (in);
        Node rn = retrieveTree(in);
        return new Node(ln, rn);
    }else{
        if(c=='\\'){
            c=in.readChar();
        }
        return new Node(c,0);
    }
}

*/

```



```
private static String codeTree(Node n){ //metodo tradizionale
```

```
    Stack<Node> stack= new Stack<Node>();
```

```
    stack.push(n);
```

```
    String htcode = "";
```

```
    do{
```

```
        n=stack.pop();
```

```
        if(n.isLeaf()){
```

```
            char c = n.symbol();
```

```
            if((c=="\\")||(c=="@")){
```

```
                htcode=htcode + "\\" + c;
```

```
            }else{
```

```
                htcode=htcode + c;
```

```
            }
```

```
        }else{
```

```
            htcode=htcode + "@";
```

```
            stack.push(n.right());
```

```
            stack.push(n.left());
```

```
        }
```

```
    }while(!stack.empty());
```

```
    return htcode;
```

```
}
```

```

public static void compress(String src, String dst){

    int[] freq = charFreq(src);

    Node tree= huffmanTree(freq);

    String htcode=codeTree(tree);

    String[] tab = huffmanTable(tree);


    InputTextFile in = new InputTextFile(src); //per provare a capire cosa fa guarda
    EsempioO

    OutputTextFile out = new OutputTextFile(dst); //per provare a capire cosa fa guarda
    EsempioO


    out.writeTextLine("" + tree.weight());

    out.writeTextLine(htcode);


    while(in.textAvailable()){

        char c= in.readChar();

        out.writeCode(tab[c]);

    }


    in.close();

    out.close();

}


public static void decompress(String src, String dst){


    InputTextFile in = new InputTextFile(src); //per provare a capire cosa fa guarda
    EsempioO

```

OutputTextFile out = new OutputTextFile(dst); //per provare a capire cosa fa guarda
Esempio

```
String num= in.readLine();
```

```
int size=Integer.parseInt(num);
```

```
Node root=retrieveTree(in);
```

```
String dummy = in.readLine();
```

```
for(int i=0; i<size; i=i+1){
```

```
    char c= retrieveChar(root, in );
```

```
    out.writeChar(c);
```

```
}
```

```
in.close();
```

```
out.close();
```

```
}
```

```
private static Node retrieveTree( InputTextFile in){ //versione nuova
```

```
    Stack<EvolvFrame> stack= new Stack<EvolvFrame>();
```

```
    stack.push(new EvolvFrame());
```

```
    Node n = null;
```

```
    do{
```

```
        EvolvFrame f = stack.peek();
```

```
        switch(f.state){
```

```

case 0: {
    char c= in.readChar();
    if(c=='@'){
        f.state=1;
        stack.push(new EvolvFrame());
    }else{
        if(c=='\'){
            c=in.readChar();
        }
        n = new Node (c, 0);
        stack.pop();
    }
    break;
}

case 1: {
    f.lft = n;
    f.state = 2;
    stack.push(new EvolvFrame());
    break;
}

case 2: {
    f.rgt=n;
    n = new Node(f.lft, f.rgt);
    stack.pop();
    break;
}
}

```

```

    }while (!stack.empty());

    return n;
}

private static char retrieveChar(Node n, InputTextFile in){
    do {
        int bit = in.readBit();

        if(bit == 0)
            n = n.left();
        else
            n = n.right();

    }while( !n.isLeaf());

    return n.symbol();
}
} //class Huffman

```